Reproducibility Sheet
REPRO_SL_dbaskaran6

Divyesh Baskaran

(a) Overleaf project (edit link)

Edit link:

https://www.overleaf.com/project/6ab951102cb870b10bb0a332/share#4f7919079198a96ed2ec65e2cd54467ba5ea4b4a2881e871

(b) GitHub commit hash

Final push commit (`master`, matches `origin/master`):

`3226bd38087c945ec11f5510142e4d18151f366a`

Commit message: `Final SL Push`

(c) Exact run instructions (standard Linux machine)

Environment setup

```
git clone https://github.gatech.edu/gt-omscs-ml/cs-7641-2026-fall-dbaskaran6.git
cd cs-7641-2026-fall-dbaskaran6/sl-report

python3.12 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

`requirements.txt` (unpinned) resolves, at time of submission, to:

Package	Version
Python	3.12.13
numpy	2.5.3
pandas	3.0.6
scikit-learn	1.9.1
matplotlib	3.11.2
torch	2.14.0
jupyter	(latest resolved by pip at install time)
nbformat	(latest resolved by pip at install time)

Data path

Place the raw BRFSS CSV at:

`sl-report/data/diabetes_012_health_indicators_BRFSS2015.csv`

(source: `Diabetes Health Indicators BRFSS 2015.zip` — unzip into `sl-report/data/`)

Commands (run in order, from `sl-report/`)

```
jupyter nbconvert --to notebook --execute --inplace 01_eda.ipynb
jupyter nbconvert --to notebook --execute --inplace harness.ipynb
jupyter nbconvert --to notebook --execute --inplace 02_decision_tree.ipynb
jupyter nbconvert --to notebook --execute --inplace 03_knn.ipynb
jupyter nbconvert --to notebook --execute --inplace 04_svm.ipynb
jupyter nbconvert --to notebook --execute --inplace 05_nn_sklearn.ipynb
jupyter nbconvert --to notebook --execute --inplace 06_nn_pytorch.ipynb
```

`01_eda.ipynb` writes `data/sample_20k.csv` and `results/sample_row_indices.npy`. `harness.ipynb` writes `results/train_idx.npy` and `results/test_idx.npy`, which every algorithm notebook loads via `common.py::load_split()` to reconstruct the identical 80/20 train/test split.

Random seeds

A single seed is used everywhere for determinism: `SEED = 42`, set in:

- `01_eda.ipynb` (stratified 20k sampling, `train_test_split(..., random_state=SEED)`)
- `harness.ipynb` (train/test split, `StratifiedKFold(..., random_state=SEED)`)
- `common.py` (`CV = StratifiedKFold(n_splits=5, shuffle=True, random_state=SEED)`, shared by every algorithm notebook)

(d) EDA summaries

Full data

$n = 253{,}680$ rows, 22 columns, all `float64`, 0 missing values, 23,899 exact duplicate rows kept (see "sampling procedure" below for why).

`GenHlth` target distribution (1 = Excellent … 5 = Poor):

Class	Count	Share (%)
1	45,299	17.86
2	89,084	35.12
3	75,646	29.82
4	31,570	12.44
5	12,081	4.76

Feature groups (21 predictors, `GenHlth` excluded):

- **binary (14):** HighBP, HighChol, CholCheck, Smoker, Stroke, HeartDiseaseorAttack, PhysActivity, Fruits, Veggies, HvyAlcoholConsump, AnyHealthcare, NoDocbcCost, DiffWalk, Sex
- **count (2):** MentHlth, PhysHlth (days out of last 30, bounded 0–30, right-skewed)
- **numeric (1):** BMI
- **ordinal (3):** Age, Education, Income
- **other (1):** Diabetes_012 (treated as ordinal: 0 = no diabetes, 1 = prediabetes, 2 = diabetes)

Sampled data

$n = 20{,}000$ rows, stratified on `GenHlth`.

Class	Full share (%)	Sample count	Sample share (%)	Abs. diff (pp)
1	17.86	3,571	17.85	0.01
2	35.12	7,023	35.12	0.00
3	29.82	5,964	29.82	0.00
4	12.44	2,489	12.44	0.00
5	4.76	953	4.76	0.00

Max shift in any feature's mean, full data vs. sample (top 5, all negligible):

Feature	Abs. mean shift
Age	0.0365
PhysHlth	0.0350
Income	0.0186
BMI	0.0135
MentHlth	0.0064

Rarest class after sampling is class 5 (953 rows); projected after an 80/20 split: 762 train / 191 test rows — enough to get a stable per-class F1 estimate.

(e) Sampling procedure

1. Load the full BRFSS 2015 CSV (253,680 rows, 22 columns, `float64`, 0 missing values). 23,899 exact duplicate rows are **kept** — dropping them would disproportionately strip the rarer `GenHlth` classes (1–2), which is exactly the distortion stratification is meant to prevent.
2. Draw a stratified sample of `N_SAMPLE = 20,000` rows using `sklearn.model_selection.train_test_split(idx_all, train_size=20_000, stratify=df["GenHlth"], random_state=42)`, stratifying on the `GenHlth` target so each class keeps its full-data proportion by construction.
3. Sort the sampled indices and re-index (`reset_index(drop=True)`) before saving to `data/sample_20k.csv`; the raw (unsorted) indices are also saved to `results/sample_row_indices.npy` for traceability back to the full dataset.

4. Validate the sample two ways: (i) side-by-side full-vs-sample class-proportion table (max abs. diff 0.01 pp, see table above), and (ii) max shift in any feature's mean, full data vs. sample (max 0.0365, on `Age`), as a compact check that sampling didn't distort feature relationships.

5. This `data/sample_20k.csv` is the single input to `harness.ipynb`, which performs one further stratified 80/20 train/test split (`test_size=0.20, stratify=y, random_state=42`) — touched exactly once per model, at final evaluation only.